```
    if (|P| <= n0) adhoc(P);
    divide P into smaller subinstances P1,P2,…,Pk;
    for (i=1; i <= k; i++)
      yi = divide-and-conquer(Pi);
    return merge(y1, y2, …, yk);
}
```

其中，|P|表示问题 P 的规模，n0 为一阈值，表示当问题 P 的规模不超过 n0 时，问题容易解出，不必再继续分解。adhoc(P)是该分治法中的基本子算法，用于直接解小规模的问题 P。当 P 的规模不超过 n0 时，直接用算法 adhoc(P)求解。算法 merge(y1, y2, …, yk)是该分治法中的合并子算法，用于将 P 的子问题 P1, P2, …, Pk 的解 y1, y2, …, yk 合并为 P 的解。

根据分治法的分割原则，应把原问题分为多少个子问题才较适宜？每个子问题是否规模相同或怎样才为适当？这些问题很难予以肯定回答。但人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同，即将一个问题分成大小相等的 k 个子问题的处理方法是行之有效的。许多问题可以取 k=2。这种使子问题规模大致相等的做法出自一种平衡（balancing）子问题的思想，几乎总是比子问题规模不等的做法要好。

从分治法的一般设计模式可以看出，用它设计出的程序一般是递归算法，因此分治法的计算效率通常可以用递归方程来进行分析。一个分治法将规模为 n 的问题分成 k 个规模为 n/m 的子问题去解。为方便起见，设分解阈值 n0 为 1，且 adhoc 解规模为 1 的问题耗费 1 单位时间。另外，将原问题分解为 k 个子问题及用 merge 将 k 个子问题的解合并为原问题的解需用 $f(n)$单位时间。如果用 $T(n)$表示该分治法 divide-and-conquer(P)解规模为|P|=n 的问题所需的计算时间，则

$$T(n)=\begin{cases}O(1) & n=1\\ kT(n/m)+f(n) & n>1\end{cases}$$

下面讨论如何解这个与分治法有密切关系的递归方程。通常可以用展开递归式的方法来解这类递归方程，反复代入求解得

$$T(n)=n^{\log_m k}+\sum_{j=0}^{\log_m n-1}k^j f(n/m^j)$$

注意，递归方程及其解只给出 n 等于 m 的方幂时 $T(n)$的值，但是如果 $T(n)$足够平滑，由 n 等于 m 的方幂时 $T(n)$的值可以估计 $T(n)$的增长速度。通常，可以假定 $T(n)$是单调上升的。

另外，在分析分治法的计算效率时，通常得到的是如下递归不等式：

$$T(n)\leqslant\begin{cases}O(1) & n=n_0\\ kT(n/m)+f(n) & n>n_0\end{cases}$$

在讨论最坏情况下的计算时间复杂度时，用等号（=）还是用小于或等于号（≤）是没有本质区别的。

以上讨论的是分治法的基本思想和一般原则。下面用一些具体例子来说明如何针对具体问题用分治思想来设计有效算法。

2.3 二分搜索技术

二分搜索算法是运用分治策略的典型例子。给定已排好序的 n 个元素 $a[0:n-1]$，现要在这 n 个元素中找出一特定元素 x。首先较容易想到的是用顺序搜索方法，逐个比较 $a[0:n-1]$中元素，直至找出元素 x 或搜索遍整个数组后确定 x 不在其中。这个方法没有很好地利用 n 个元素已排好序这个条件，因此在最坏情况下，顺序搜索方法需要 $O(n)$次比较。

二分搜索方法充分利用了元素间的次序关系，采用分治策略，可在最坏情况下用 $O(\log n)$时间完成搜索任务。二分搜索算法的基本思想是，将 n 个元素分成个数大致相同的两半，取 $a[n/2]$与 x 进行比较。如果 $x=a[n/2]$，则找到 x，算法终止；如果 $x<a[n/2]$，则只在数组 a 的左半部继续搜索 x；如果 $x>a[n/2]$，则只在数组 a 的右半部继续搜索 x。具体算法可描述如下：

```
template<class Type>
int BinarySearch(Type a[], const Type& x, int n)          // 在 a[0]<=a[1]<=...<=a[n-1]中搜索 x
{
  // 找到 x 时返回其在数组中的位置，否则返回-1
  int  left = 0; int  right = n-1;
  while (left <= right) {
    int  middle = (left+right)/2;
    if (x == a[middle]) return middle;
    if (x > a[middle]) left = middle+1;
    else right = middle-1;
  }
  return-1;                                          // 未找到 x
}
```

容易看出，每执行一次算法的 while 循环，待搜索数组的大小减小一半。因此，在最坏情况下，while 循环被执行了 $O(\log n)$次。循环体内运算需要 $O(1)$时间，因此整个算法在最坏情况下的计算时间复杂性为 $O(\log n)$。

二分搜索算法的思想易于理解，但是要写一个正确的二分搜索算法也不是一件简单的事。Knuth 在他的著作 *The Art of Computer Programming : Sorting and Searching* 中提到，第一个二分搜索算法早在 1946 年就出现了，但是第一个完全正确的二分搜索算法直到 1962 年才出现。

2.4 大整数的乘法

通常，在分析算法的计算复杂性时，都将加法和乘法运算看作基本运算来处理，即将执行一次加法或乘法运算所需的计算时间看作一个仅取决于计算机硬件处理速度的常数。这个假定仅在参加运算的整数能在计算机硬件对整数的表示范围内直接处理时才是合理的。然而，在某些情况下需要处理很大的整数，无法在计算机硬件能直接表示的整数范围内进行处理。若用浮点数来表示它，则只能近似地表示它的大小，计算结果中的有效数字也受到限制。若要精确地表示大整数并在计算结果中要求精确地得到所有位数上的数字，就必须用软件的方法来实现大整数的算术运算。

设 X 和 Y 都是 n 位二进制整数，现在要计算它们的乘积 XY。可以用小学所学的方法来设计一个计算乘积 XY 的算法，但是这样做计算步骤太多，效率较低。如果将每两个一位数的乘法或加法看作一步运算，那么这种方法要进行 $O(n^2)$步运算才能求出乘积 XY。下面用分治法来设计更有效的大整数乘积算法。

将 n 位二进制整数 X 和 Y 都分为 2 段，每段的长为 $n/2$ 位（为叙述简单，假设 n 是 2 的幂），如图 2-3 所示。

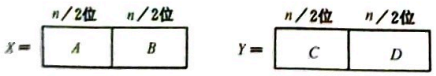

图 2-3 大整数 X 和 Y 的分段

由此，$X=A\times 2^{n/2}+B$，$Y=C\times 2^{n/2}+D$。这样，X 和 Y 的乘积为

$$XY=(A\times 2^{n/2}+B)(C\times 2^{n/2}+D)=AC\times 2^{n}+(AD+BC)\times 2^{n/2}+BD$$

如果按此式计算 XY，必须进行 4 次 $n/2$ 位整数的乘法（AC、AD、BC 和 BD）、3 次不超过 $2n$ 位的整数加法（分别对应式中的“+”），以及 2 次移位（分别对应式中的乘 2^n 和乘 $2^{n/2}$）。所有这些加法和移位共用 $O(n)$步运算。设 $T(n)$是 2 个 n 位整数相乘所需的运算总数，则

$$T(n)=\begin{cases}O(1) & n=1\\ 4T(n/2)+O(n) & n>1\end{cases}$$

由此可得 $T(n)=O(n^2)$。因此，直接用此式来计算 X 和 Y 的乘积并不比小学生的方法更有效。要想改进算法的计算复杂性，必须减少乘法次数。下面把 XY 写成另一种形式：

$$XY=AC\times 2^{n}+((A-B)(D-C)+AC+BD)\times 2^{n/2}+BD$$

此式看起来似乎复杂些，但仅需做 3 次 $n/2$ 位整数的乘法（AC、BD 和$(A-B)(D-C)$）、6 次加减法和 2 次移位。由此可得

$$T(n)=\begin{cases}O(1) & n=1\\ 3T(n/2)+O(n) & n>1\end{cases}$$

容易求得，其解为 $T(n)=O(n^{\log 3})=O(n^{1.59})$。这是一个较大的改进。

上述二进制大整数乘法同样可应用于十进制大整数的乘法以减少乘法次数，提高算法效率。如果将一个大整数分成 3 段或 4 段做乘法，计算复杂性会发生什么变化呢？是否优于分成 2 段来做乘法？读者可以通过有关练习得到明确的结论。

2.5 斯特拉森矩阵乘法

矩阵乘法是线性代数中最常见的问题之一，在数值计算中有广泛的应用。设 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 是两个 $n\times n$ 矩阵，它们的乘积 $\boldsymbol{AB}$ 同样是一个 $n\times n$ 矩阵。$\boldsymbol{A}$ 和 $\boldsymbol{B}$ 的乘积矩阵 $\boldsymbol{C}$ 中元素 c_{ij} 定义为

$$c_{ij}=\sum_{k=1}^{n}a_{ik}b_{kj}$$

若依此定义来计算 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 的乘积矩阵 $\boldsymbol{C}$，则每计算 $\boldsymbol{C}$ 的一个元素 c_{ij}，需要做 n 次乘法和 $n-1$ 次加法。因此，求出矩阵 $\boldsymbol{C}$ 的 n^2 个元素所需的计算时间为 $O(n^3)$。

20 世纪 60 年代末，斯特拉森（Strassen）采用了类似于在大整数乘法中用过的分治技术，将计算 2 个 n 阶矩阵乘积所需的计算时间改进到 $O(n^{\log 7})=O(n^{2.81})$。其基本思想还是使用分治法。

首先，仍假设 n 是 2 的幂。将矩阵 $\boldsymbol{A}$、$\boldsymbol{B}$ 和 $\boldsymbol{C}$ 中每个矩阵都分块成 4 个大小相等的子矩阵，每个子矩阵都是 $n/2\times n/2$ 的方阵。由此可将方程 $\boldsymbol{C}=\boldsymbol{AB}$ 重写为

$$\begin{bmatrix}\boldsymbol{C}_{11} & \boldsymbol{C}_{12}\\ \boldsymbol{C}_{21} & \boldsymbol{C}_{22}\end{bmatrix}=\begin{bmatrix}\boldsymbol{A}_{11} & \boldsymbol{A}_{12}\\ \boldsymbol{A}_{21} & \boldsymbol{A}_{22}\end{bmatrix}\begin{bmatrix}\boldsymbol{B}_{11} & \boldsymbol{B}_{12}\\ \boldsymbol{B}_{21} & \boldsymbol{B}_{22}\end{bmatrix}$$

由此可得

$$\boldsymbol{C}_{11}=\boldsymbol{A}_{11}\boldsymbol{B}_{11}+\boldsymbol{A}_{12}\boldsymbol{B}_{21}$$
$$\boldsymbol{C}_{12}=\boldsymbol{A}_{11}\boldsymbol{B}_{12}+\boldsymbol{A}_{12}\boldsymbol{B}_{22}$$
$$\boldsymbol{C}_{21}=\boldsymbol{A}_{21}\boldsymbol{B}_{11}+\boldsymbol{A}_{22}\boldsymbol{B}_{21}$$
$$\boldsymbol{C}_{22}=\boldsymbol{A}_{21}\boldsymbol{B}_{12}+\boldsymbol{A}_{22}\boldsymbol{B}_{22}$$

如果 $n=2$，则 2 个 2 阶方阵的乘积可以直接计算出来，共需 8 次乘法和 4 次加法。当子矩阵的阶大于 2 时，为求 2 个子矩阵的积，可以继续将子矩阵分块，直到子矩阵的阶降为 2。由此产生了分治降阶的递归算法。依此算法，计算 2 个 n 阶方阵的乘积转化为计算 8 个 $n/2$ 阶方阵的乘积和 4 个 $n/2$ 阶方阵的加法。2 个 $n/2\times n/2$ 矩阵的加法显然可以在 $O(n^2)$时间内完成。因此，上述分治法的计算时间耗费 $T(n)$应满足

$$T(n)=\begin{cases}O(1) & n=2\\ 8T(n/2)+O(n^2) & n>2\end{cases}$$

这个递归方程的解仍然是 $T(n)=O(n^3)$。因此，该方法并不比用原始定义直接计算更有效。究其原因，是由于该方法并没有减少矩阵的乘法次数。而矩阵乘法耗费的时间要比矩阵加（减）法耗费的时间多得多，要想改进矩阵乘法的计算时间复杂性，必须减少乘法运算。

按照上述分治法的思想可以看出，要想减少乘法运算次数，关键在于计算 2 个 2 阶方阵的乘积时，能否用少于 8 次的乘法运算。Strassen 提出了一种新的算法来计算 2 个 2 阶方阵的乘积。该算法只用了 7 次乘法运算，但增加了加减法的运算次数。这 7 次乘法是

$$M_1=A_{11}(B_{12}-B_{22})$$
$$M_2=(A_{11}+A_{12})B_{22}$$
$$M_3=(A_{21}+A_{22})B_{11}$$
$$M_4=A_{22}(B_{21}-B_{11})$$
$$M_5=(A_{11}+A_{22})(B_{11}+B_{22})$$
$$M_6=(A_{12}-A_{22})(B_{21}+B_{22})$$
$$M_7=(A_{11}-A_{21})(B_{11}+B_{12})$$

做了这 7 次乘法后，再做若干次加减法，就可以得到

$$C_{11}=M_5+M_4-M_2+M_6$$
$$C_{12}=M_1+M_2$$
$$C_{21}=M_3+M_4$$
$$C_{22}=M_5+M_1-M_3-M_7$$

以上计算的正确性容易验证。

Strassen 矩阵乘法中用了 7 次对于 $n/2$ 阶矩阵乘积的递归调用和 18 次 $n/2$ 阶矩阵的加减运算。由此可知，该算法所需的计算时间 $T(n)$满足如下递归方程：

$$T(n)=\begin{cases}O(1) & n=2\\ 7T(n/2)+O(n^2) & n>2\end{cases}$$

解此递归方程得 $T(n)=O(n^{\log 7})\approx O(n^{2.81})$。由此可见，Strassen 矩阵乘法的计算时间复杂性比普通矩阵乘法有较大改进。

有人曾列举了计算 2 个 2×2 阶矩阵乘法的 36 种方法。但所有的方法都至少做 7 次乘法。除非能找到一种计算 2 阶方阵乘积的算法，使乘法的计算次数少于 7 次，计算矩阵乘积的计算时间下界才有可能低于 $O(n^{2.81})$。但是 Hopcroft 和 Kerr 已经证明（1971 年），计算 2 个 2×2 矩阵的乘积，7 次乘法是必要的。因此，要想进一步改进矩阵乘法的时间复杂性，就不能再基于计算 2×2 矩阵的 7 次乘法这样的方法了，或许应当研究 3×3 或 5×5 矩阵的更好算法。在 Strassen 之后有许多算法改进了矩阵乘法的计算时间复杂性，目前最好的计算时间上界是 $O(n^{2.376})$，所知的矩阵乘法的最好下界仍是它的平凡下界 $\Omega(n^2)$。因此，到目前为止还无法确切知道矩阵乘法的时间复杂性。关于这一研究课题还有许多工作可做。

2.6 棋盘覆盖

在一个 $2^k\times2^k$ 个方格组成的棋盘中，若恰有一个方格与其他方格不同，则称该方格为一特殊方格，且称该棋盘为一特殊棋盘。显然，特殊方格在棋盘上出现的位置有 4^k 种情形。因而对任何 $k\geq0$，有 4^k 种特殊棋盘。图 2-4 中的特殊棋盘是 $k=2$ 时 16 个特殊棋盘中的一个。

在棋盘覆盖问题中，要用图 2-5 所示的 4 种不同形态的 L 型骨牌覆盖一个给定的特殊棋盘上除特殊方格以外的所有方格，且任何 2 个 L 型骨牌不得重叠覆盖。易知，在任何一个 $2^k\times2^k$ 的棋盘覆盖中，用到的 L 型骨牌个数恰为$(4^k-1)/3$。

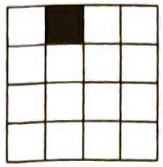

图 2-4　$k=2$ 时的一个特殊棋盘

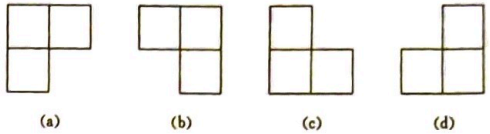

图 2-5　4 种不同形态的 L 型骨牌

用分治策略，可以设计解棋盘覆盖问题的一个简捷的算法。当 $k>0$ 时，将 $2^k\times2^k$ 棋盘分割为 4 个 $2^{k-1}\times2^{k-1}$ 子棋盘，如图 2-6(a)所示。特殊方格必位于 4 个较小子棋盘之一中，其余 3 个子棋盘中无特殊方格。为了将这 3 个无特殊方格的子棋盘转化为特殊棋盘，可以用一个 L 型骨牌覆盖这 3 个较小棋盘的会合处，如图 2-6(b)所示，这 3 个子棋盘上被 L 型骨牌覆盖的方格就成为该棋盘上的特殊方格，从而将原问题转化为 4 个较小规模的棋盘覆盖问题。递归地使用这种分割，直至棋盘简化为 1×1 棋盘。

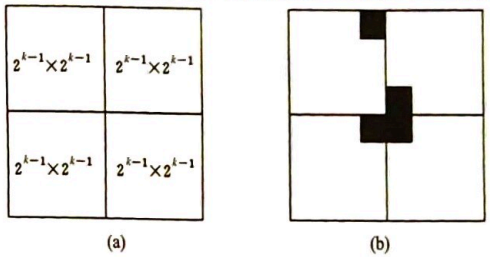

图 2-6　棋盘分割

实现这种分治策略的算法 ChessBoard 如下：

```
void ChessBoard(int tr, int tc, int dr, int dc, int size)
{
   if (size == 1)
      return;
   int  t=tile++,                                    // L 型骨牌号
   s=size/2;                                         // 分割棋盘
   // 覆盖左上角子棋盘
   if (dr <tr+s && dc<tc+s)                          // 特殊方格在此棋盘中
      ChessBoard(tr, tc, dr, dc, s);
   else {                                            // 此棋盘中无特殊方格
      Board[tr+s-1][tc+s-1]=t;                       // 用 t 号 L 型骨牌覆盖右下角
      ChessBoard(tr, tc, tr+s-1, tc+s-1, s);         // 覆盖其余方格
   }
   if (dr < tr+s&& dc >= tc+s)                       // 覆盖右上角子棋盘
      ChessBoard(tr, tc+s, dr, dc, s);               // 特殊方格在此棋盘中
   else {                                            // 此棋盘中无特殊方格
      Board[tr+s-1][tc+s] = t;                       // 用 t 号 L 型骨牌覆盖左下角
      ChessBoard(tr, tc+s, tr+s-1, tc+s, s);         // 覆盖其余方格
   }
   if (dr >= tr+s && dc < tc+s)                      // 覆盖左下角子棋盘
      ChessBoard(tr+s, tc, dr, dc, s);               // 特殊方格在此棋盘中
```

```
    else {                                          // 用 t 号 L 型骨牌覆盖右上角
      Board[tr+s][tc+s-1] = t;
      ChessBoard(tr+s, tc, tr+s, tc+s-1, s);        // 覆盖其余方格

    if (dr >= tr+s && dc >= tc+s)                   // 覆盖右下角子棋盘
      ChessBoard(tr+s, tc+s, dr, dc, s);            // 特殊方格在此棋盘中
    else {                                          // 用 t 号 L 型骨牌覆盖左上角
      Board[tr+s][tc+s] = t;
      ChessBoard(tr+s, tc+s, tr+s, tc+s, s);        // 覆盖其余方格
    }
  }
```

上述算法中用一个二维整型数组 Board 表示棋盘。Board[0][0]是棋盘的左上角方格。tile 是算法中的一个全局整型变量，用来表示 L 型骨牌的编号，其初始值为 0。算法的输入参数是：

tr：棋盘左上角方格的行号；　　dc：特殊方格所在的列号；

tc：棋盘左上角方格的列号；　　size：size=2^k，棋盘规格为 $2^k \times 2^k$；

dr：特殊方格所在的行号。

设 $T(k)$是算法 ChessBoard 覆盖一个 $2^k \times 2^k$ 棋盘所需的时间，则从算法的分治策略可知，$T(k)$满足如下递归方程

$$T(k)=\begin{cases} O(1) & k=0 \\ 4T(k-1)+O(1) & k>0 \end{cases}$$

解此递归方程可得 $T(k)=O(4^k)$。由于覆盖一个 $2^k \times 2^k$ 棋盘所需的 L 型骨牌个数为$(4^k-1)/3$，故算法 ChessBoard 是一个在渐近意义下最优的算法。

2.7 合并排序

合并排序算法是用分治策略实现对 n 个元素进行排序的算法，其基本思想是：将待排序元素分成大小大致相同的 2 个子集合，分别对 2 个子集合进行排序，最终将排好序的子集合合并成要求的排好序的集合。合并排序算法可递归地描述如下：

```
template<class Type>
void MergeSort(Type a[], int left, int right)
{
  if (left < right){                                // 至少有 2 个元素
    int  i = (left+right)/2;                        // 取中点
    MergeSort(a, left, i);
    MergeSort(a, i+1, right);
    Merge(a, b, left, i, right);                    // 合并到数组 b
    Copy(a, b, left, right);                        // 复制回数组 a
  }
}
```

其中，算法 Merge 合并 2 个排好序的数组段到一个新的数组 b 中，然后由 Copy 将合并后的数组段再复制回数组 a 中。Merge 和 Copy 显然可在 $O(n)$时间内完成，因此合并排序算法对 n 个元素进行排序，在最坏情况下所需的计算时间 $T(n)$满足

$$T(n)=\begin{cases} O(1) & n \leqslant 1 \\ 2T(n/2)+O(n) & n>1 \end{cases}$$

解此递归方程可得 $T(n)=O(n\log n)$。由于排序问题的计算时间下界为 $\Omega(n\log n)$，故合并排序算法是